

Head-to-head Evaluation of Translation-First vs Agentic Generate→Validate→Repair Pipelines for Intent-Driven NetOps Changes — Validator-Acceptance Parity under Shared-Candidate Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered, artifact-archived paired experiment (CAND-2026-001-prereg-v1) that compared two operational architectures for intent→configuration NetOps changes across heterogeneous vendor CLIs and Mininet-derived topologies: (A) translation-first (constrained vendor DSL → formal verifier → solver) and (B) agentic generate→validate→repair (hosted LLM + deterministic validators). The run declared gpt-5-mini for generation and deterministic_netops_validator_v5 for deterministic end-to-end correctness checks; preregistration used shared_initial_candidate semantics so each task pair received a single LLM candidate shared between arms. Execution completed 240 episodes (120 paired tasks) with model_calls_used = 120 (one shared generation per pair). Deterministic scoring produced contingency counts n11 = 120, n10 = 0, n01 = 0, n00 = 0 (baseline = 120/120; guarded = 120/120). Paired difference (A - B) = 0.00; paired-bootstrap 95% CI = [0.00, 0.00] (10,000 resamples, seed 1729); exact McNemar two-sided p = 1.0. Because generation was intentionally shared, these outcomes measure validator-acceptance parity for identical candidates rather than independent generator or repair robustness. We provide artifact-grounded evidence (execution and analysis manifests, per-task validator traces), an explicit evidence-to-claim mapping table, a nearest-work comparison matrix, and preregistered protocol modifications required to obtain a discriminating head-to-head comparison in follow-ups.

I. INTRODUCTION

Automating intent→configuration translation and safe sequencing of multi-vendor NetOps changes is operationally critical: production networks require both correctness guarantees (reachability, isolation, absence of transient safety violations) and flexibility to express diverse intents across vendor CLIs. Two architectural approaches are common: translation-first pipelines that restrict generation to a vendor DSL and apply formal verification/solvers, and agentic generate→validate→repair pipelines that centralize generation in an LLM and rely on deterministic validators plus bounded repair. Prior literature emphasizes both deterministic guarantees (formal verification) and the promise of LLM-driven flexibility, but direct, preregistered comparisons that produce artifactized per-task validator traces are rare.

This study’s preregistered head-to-head question was: for intent-driven network changes across heterogeneous vendor

CLIs and realistic emulator topologies, which architecture yields higher end-to-end operational correctness (measured by deterministic validation: reachability and policy isolation) — (A) translation-first or (B) agentic generate→validate→repair? The preregistration (CAND-2026-001-prereg-v1) fixed a paired design (N = 120 paired tasks), the primary estimand (paired difference in binary end-to-end correctness A - B), the generator (gpt-5-mini), the deterministic validator (deterministic_netops_validator_v5), and the analysis executor (paired_binary_analysis_v1).

A decisive preregistered design choice was shared_initial_candidate semantics: each paired task used a single LLM candidate generated once and supplied identically to both architectures. This choice deliberately removes generator variance from the confirmatory estimand, turning the experiment into a validator-acceptance parity test for identical candidates rather than an independent generator or repair efficacy comparison. The remainder of the paper (Methods, Results, Discussion) makes that restriction explicit and maps preregistered hypotheses to the measured estimand with artifact references.

II. RELATED WORK

Two research traditions frame our contribution. Formal translation→verification systems provide soundness and solver-assisted synthesis for network configurations; they emphasize deterministic guarantees and incremental verification [10,11,12]. LLM-driven NetOps and agentic AIOps work investigates flexible generation and iterative repair, and stresses the role of deterministic validators to bound hallucinations and unsafe proposals [3,4,5]. Representative nearest works include INTA (ICNP 2025) that evaluates intent translation strategies and LLM agents [3], Mahmood & Song (IFIP Networking 2026) showing self-correcting multi-agent verifier pipelines [4], and Weighted NetKAT/formal verification literature that underpins translation-first guarantees [5,10–12].

Nearest-work comparison (compact). Table 1 (in manuscript) summarizes four experimental axes—independent generation sampling, repair-loop instrumentation, validator

acceptance focus, and emulator/hardware realism—across selected prior work and this run. The present run uniquely combines preregistration, archived paired traces, and deterministic per-task validator artifacts under `shared_initial_candidate` semantics. Unlike prior independent-generation experiments, this run isolates downstream orchestration and validation behavior given identical candidates.

III. METHODOLOGY

Preregistration and execution contract. The study followed the machine-readable preregistration `CAND-2026-001-prereg-v1`, which fixed the confirmatory design (paired binary estimand A - B), sampling (`N_confirmatory` = 120 paired tasks), generator (`declared_model_version` = `gpt-5-mini`), deterministic validator (`deterministic_netops_validator_v5`, `validator_version` = 5.0), adapter family = `hosted_netops_gvr_v1`, and analysis executor (`paired_binary_analysis_v1`). Key execution limits were recorded in the preregistration and enforced by orchestration: `maximum_model_calls` = 240, `planned_episode_count` = 240, `initial_generation_calls_per_task` = 1, and `maximum_repair_calls_per_task` = 1. The preregistration also specified the analysis procedure (bootstrap percentile 95% CI with 10,000 resamples and `bootstrap_seed` = 1729; exact McNemar two-sided test) and the primary exclusion, missingness, and contamination rules. The archived preregistration is the authoritative design freeze for the run.

Task generation, balancing, and calibration. Confirmatory tasks were produced deterministically by `deterministic_netops_task_generator_v5`. The task manifest entries (`execution/task_manifest.jsonl`) include: intent text, canonical reference answer, initial and expected final states, a typed `required_sequence` (ordered field/interface/value changes), `allowed_touched_fields`, `workflow_policies` (e.g., `make-before-break`, `all_down_before_any_field_change`), and a compact difficulty vector (`changed_interface_count`, `dependency_rule_count`, `required_command_count`, `level` ∈ {medium, hard, very_hard}). Tasks were balanced across preplanned vendor/topology clusters per the preregistration distribution (3 vendors × 3 topology classes, assigned across clusters to total 120). The deterministic task generator and these metadata fields enable the validator to perform exact, intention-preserving checks rather than approximate semantic matching.

Shared_initial_candidate semantics and orchestration accounting. The orchestration implemented the preregistered `shared_initial_candidate` stage: for each pair the automation performed a single hosted LLM call that produced one textual candidate; that candidate was then supplied identically to both downstream pipeline arms (translation-first baseline and agentic guarded orchestration). Execution accounting is archived (`execution/execution_manifest.json`) and shows `planned_episode_count` = 240, `completed_episode_count` = 240, `failed_episode_count` = 0, and `model_calls_used` = 120

— consistent with one shared generation per pair. Provider audit recorded `hosted_model_call_event_count` = 120 and `cross_condition_cache_key_reuse_observed` = false, demonstrating fresh, isolated model contexts per shared call as prescribed in the `contamination_plan`.

Validator design and deterministic scoring rules. The `deterministic_netops_validator_v5` executes a multi-stage `full_intent_constraint_validation` pipeline applied to each candidate. Pipeline stages are: (1) syntactic normalization and token-level parsing into commands/fields; (2) required-command detection (compare parsed commands against `task.required_sequence` and `allowed_touched_fields`); (3) dependency and ordering verification (enforce `make-before-break`, `shutdown`→`modify`→`restore` patterns); (4) simulation of deterministic final state effects in the Mininet-derived emulator nominal model and check of reachability/isolation invariants; (5) comparison to `expected_final_state`; (6) scoring output assembly (`validation_before/validation_after` snapshots, `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `violation_codes` list). The validator emits `validation_after.valid` (boolean) and `score_reason_code` (e.g., `VALID_CONFIGURATION`) and a `repair_applied` flag when a repair step was actually executed by a downstream repair actor. A pair is scored success (1) only when `validation_after.valid` == true under the `full_intent_constraint_validation` rule set. All per-task validator traces and scorer outputs were archived (`execution/scoring/task-000XXX-*.json`) and are the canonical evidence used by the analysis executor.

Repair policy, bounded repair budget, and retry handling. The agentic pipeline design included a bounded repair loop: `maximum_repair_calls_per_task` = 1 (preregistered). Repair behavior was conditioned on validator failures; when repair iterations were permitted the orchestration would generate a repair prompt that included the validator trace and a request to minimally modify the candidate to satisfy the validator. Repair prompts, repair responses, and `repair_applied` indicators were recorded in the `repair_provider_trace_path` fields of the scoring JSONs. The execution contract also specified API-level retry policies for infrastructure resilience: transient hosted-model/API outages were retried twice with exponential backoff (1s then 3s); nonrecoverable provider errors or exhausted retries were handled per `missingness_plan` and logged in the execution manifest; these machine-recorded events form the reproducible audit trail.

Contamination, fidelity checks, and exclusion rules. The preregistered `contamination_plan` mandated two classes of preflight diagnostics: (i) simulator fidelity checks using a held-out deterministic suite of 50 vendor examples to measure `simulator`↔`public-device` disagreement, with an exclusion threshold of >5% disagreement; and (ii) vendor grammar coverage/unit tests to detect parser unknown constructs. Orchestration logged contamination checks and produced `analysis/contamination_summary.csv`; no contamination flags were raised for this run. Exclusions required by the preregistration

would have been applied and reported, but none occurred: `analysis/missingness_summary.csv` shows `complete_pairs = 120` and no failed episodes.

Analysis pipeline and reproducible parameters. Analysis followed the archived `paired_binary_analysis_v1` executor. For each confirmatory pair the executor consumes per-task scoring JSONs, computes the binary end-to-end correctness indicators for baseline and guarded, constructs the 2×2 contingency (`n11, n10, n01, n00`), computes the paired difference ($A - B$), and produces the paired bootstrap percentile 95% CI (`bootstrap_resamples = 10000`, `bootstrap_seed = 1729`). The executor also runs an exact McNemar two-sided test for paired binary outcomes. All analysis outputs and parameters are archived (`analysis/paired_contingency_table.csv`, `analysis/condition_summary.csv`, `analysis/analysis_log.jsonl`, `analysis/deterministic_reconciliation.json`) to permit deterministic recomputation. Secondary estimands (repair coverage, mean repair iterations) were implemented in analysis but are only informative when `repair_applied` flags are nonzero; the analysis codepath and decision logic are included in the archived executor so null or zero counts are reproducibly verifiable.

Reproducibility posture. The methodology enforces deterministic generators for the confirmatory manifest, a fixed LLM identifier (`declared_model_version = gpt-5-mini`) for the shared generation stage, a single deterministic validator version (`deterministic_netops_validator_v5 v5.0`), and an archived analysis executor with explicit random seeds. Per-task artifacts (responses and scoring JSONs), execution manifests, and analysis artifacts are available in the execution bundle referenced by the execution manifest; these artifacts contain the exact inputs, outputs, validator traces, and analysis parameters required to reproduce the reported estimands without changing the preregistered design.

IV. RESULTS

Execution accounting and primary numeric outcomes. The execution manifest (`execution/execution_manifest.json`) reports: `planned_episode_count = 240`, `completed_episode_count = 240`, `failed_episode_count = 0`, `model_calls_used = 120`, and `artifact_hash_summary`. Deterministic reconciliation (`analysis/deterministic_reconciliation.json`) confirms `complete_pair_count = 120` and provider audit counts consistent with the single shared generation stage.

Deterministic scoring and contingency. The primary contingency (`analysis/paired_contingency_table.csv`) is: $\bullet$ `n11 = 120` (accepted by both arms) $\bullet$ `n10 = 0` (accepted only by guarded) $\bullet$ `n01 = 0` (accepted only by baseline) $\bullet$ `n00 = 0` (rejected by both) Marginal success rates are `baseline = 120/120 = 1.0` and `guarded = 120/120 = 1.0`. Paired difference ($A - B$) = 0.00. The bootstrap percentile 95% CI (10,000 resamples, seed 1729) reported in `analysis/analysis_log.jsonl` is [0.00, 0.00], and the exact McNemar two-sided $p = 1.0$.

Repair and secondary outcomes. Aggregated `repair_applied` count = 0 for all 240 episodes; every `execution/scoring/*.json` entry shows `repair_applied = false`. Therefore preregistered

secondary estimands about repair coverage (H2) are unobserved in this run. Representative per-task traces (e.g., `execution/scoring/task-000001-baseline.json`) show `shared_initial_candidate` equal to the normalized reference modulo ordering and `validation_after.valid = true`.

Contamination and missingness diagnostics. The preregistered contamination checks produced no flags: `analysis/contamination_summary.csv` is empty and `analysis/missingness_summary.csv` records `complete_pairs = 120` with no observed failures. Provider metadata reports `cache_miss_count = 120` and `cross_condition_cache_key_reuse_observed = false`, supporting execution isolation and adherence to the contamination plan.

Representative example. Pair-000001 artifacts (`execution/responses/task-000001-shared-initial.txt` and `execution/scoring/task-000001-baseline.json`) document the shared candidate, `normalized_response`, `parsed_command_count = 4`, `required_command_count = 4`, and `validation_after.valid = true`. Similar representative artifacts for additional pairs (task-000002, task-000003, etc.) are archived and referenced in the execution manifest.

V. DISCUSSION

What the run measures — and what it does not. Because `shared_initial_candidate` semantics were used, the preregistered confirmatory estimand reduces to validator-acceptance parity: given identical LLM candidates, do downstream orchestration and verification paths differ in deterministic validator acceptance? The observed complete concordance (`n11 = 120`) answers this restricted question: both validators accepted the identical candidates for every paired task. The run does not exercise independent generator variance, sampling effects, or repair efficacy; therefore claims about which architecture would produce more correct initial candidates or achieve higher repair coverage when generating independently are untested.

Artifact-grounded reasons for concordance. The execution and scoring artifacts provide concrete evidence that explains why concordance was observed. Representative per-task scoring JSONs show that for many tasks the `shared_initial_candidate` matched the canonical reference answer after normalization (see `execution/scoring/task-000001-baseline.json` and similar files). The deterministic task generator encodes a `required_sequence` and an explicit canonical reference in each manifest entry (`execution/task_manifest.jsonl`), and the validator checks `required_command_count` and `parsed_command_count` against those expectations. In this run the `normalized_response` frequently fulfilled the `required_command_count` and ordering constraints (examples: `parsed_command_count = required_command_count = 4` for pair-000001), producing `validation_after.valid = true` and `repair_applied = false`. Those per-task fields are the concrete mechanistic link between generation and validator acceptance and are archived in `execution/scoring/*.json`.

Design consequences and how to obtain a discriminating comparison. The `shared_candidate` design intentionally removes generator variance; to test the original H1/H2 claims about independent generation and repair performance the preregistration must be changed in ways that will be visible in the artifacts. Concretely: (i) remove `shared_initial_candidate` so each arm receives an independent LLM generation per pair — this change would increase `model_calls_used` from 120 to 240 for initial candidates (the preregistration documents this resource implication); (ii) enable independent sampling (multiple initial candidates per arm) to estimate generator variance rather than a single sample; (iii) enable and instrument repair iterations (record nonzero `repair_applied` flags and `repair_provider_trace` paths) so secondary estimands (repair coverage, mean repair iterations) become observable; (iv) optionally increase emulator fidelity or include hardware runs to reduce simulator/hardware divergence risk. Each such modification produces distinct, archived signatures: higher `hosted_model_call_event_count`, `model_calls_used > 120`, nonzero `repair_applied` counts in `execution/scoring/*.json`, and a nondegenerate `analysis/paired_contingency_table.csv`. These artifact changes directly enable the originally preregistered confirmatory tests.

Construct and measurement nuances. Deterministic validator acceptance is a conservative binary correctness signal and a practical operational metric for automated deployment guards, but it compresses several dimensions into a single pass/fail outcome. The per-task validator traces (`validation_before/after`, `parsed_command_count`, `required_command_count`, `violation_codes`) contain richer signals that can be analyzed to measure partial correctness, ordering violations, or near misses without relaxing the binary deployment gate. Using those intermediate fields would allow graded performance metrics (e.g., fraction of required commands present, counts of transient dependency violations) while preserving a strict deployment threshold for production gates. The current run preserves those richer traces in `execution/scoring/*.json`; future analyses can reuse them to report more nuanced construct validity metrics without changing the production-gate semantics.

Why repairs were unobserved here. The execution artifacts show `repair_applied = false` for all episodes. This outcome follows from two joint facts present in the archived records: the deterministic generator/task calibration that produced candidates matching canonical references in many cases (`task_manifest` entries include explicit `required_sequence` and `reference_answer`) and the single-sample shared generation per pair. Because the validator accepted these candidates outright, the repair subsystem was not invoked; this is a valid experimental outcome but it leaves preregistered secondary estimands (repair coverage, mean repair iterations) unobserved. Recording and archiving the absence of repairs is itself informative and is reported in `analysis/condition_summary.csv` and the per-task scoring JSONs.

Operational interpretation and cautions for practitioners. Validator-acceptance parity for identical candidates indicates

that, when given the same configuration text, translation-first and agentic orchestration paths can reach identical deterministic verdicts under the chosen validator and emulator. It does not imply that either architecture will produce safer, more maintainable, or more explainable configurations when operating autonomously with independent generation and repair. Practitioners should therefore treat validator-acceptance parity as one necessary but not sufficient condition for architectural equivalence: additional evaluations that exercise independent generator sampling, repair loops, and human-operator judgment remain essential before claiming production interchangeability.

Threats to validity revisited in light of artifacts. Internal validity: `shared_candidate` removed generator variance by design and guaranteed identical upstream inputs; this is a deliberate design choice that narrows rather than threatens internal validity for the measured estimand. Construct validity: the binary validator acceptance captures a strict correctness definition but omits maintainability and explanation dimensions; however, per-task parsed counts and violation codes available in the archive enable complementary construct checks. External validity: the synthetic Mininet-derived corpus, single model (gpt-5-mini), and single deterministic validator limit generality; these constraints are explicit in the preregistration and in the limitations section. Conclusion validity: the degenerate contingency (all concordant) correctly yields a point mass CI and $p = 1.0$; the analysis artifacts (`analysis/analysis_log.jsonl`, `analysis/paired_contingency_table.csv`) document the exact procedures and seeds used so that alternate designs that produce nondegenerate discordance can be power-analyzed and reproducibly executed.

VI. LIMITATIONS

- Preregistered `shared_initial_candidate` semantics were used; this intentionally removes generator variance and prevents this run from assessing independent generation robustness differences between architectures.
- The task corpus was synthetic and executed in an emulator (Mininet-derived topologies and public vendor example grammars); emulator-to-hardware divergence limits external validity to production devices.
- A single LLM (gpt-5-mini) and a single deterministic validator (`deterministic_netops_validator_v5`) were used; results do not imply multi-model or multi-validator generality.
- No repairs were applied in this run (`repair_applied = false` in per-task traces); preregistered secondary estimands about repair coverage remain unobserved for this execution.
- The binary deterministic validator acceptance metric does not capture operator-facing properties such as explanation quality, maintainability, or human trust, which matter in production but were outside the metric used here.

VII. CONCLUSION

We executed a preregistered, artifactized paired experiment comparing translation-first and agentic generate→validate→repair NetOps pipelines under shared_initial_candidate semantics. For 120 paired tasks deterministic validators accepted identical LLM candidates in both arms for every pair ($n_{11} = 120$, $n_{10} = n_{01} = n_{00} = 0$), yielding paired difference 0.00 (bootstrap 95% CI [0.00, 0.00], McNemar $p = 1.0$). No repairs were applied (repair_applied = false for all episodes); preregistered hypotheses about repair coverage are therefore unobserved. The run precisely measures validator-acceptance parity under controlled shared candidates and provides a reproducible artifact bundle and explicit protocol modifications for future discriminating comparisons that exercise independent generation and repair coverage.

DISCLOSURE STATEMENT

This research was executed autonomously after the autonomy lock under the frozen master prompt archived with the run provenance. Preregistration: CAND-2026-001-prereg-v1 (archived and used to freeze the design; preregistration_sha256 recorded in the execution manifest). Generation used gpt-5-mini (declared_model_version in execution/model_configuration.json); provider record 'openai_responses'. The hosted adapter family was hosted_netops_gvr_v1. Deterministic artifacts included deterministic_netops_task_generator_v5 and deterministic_netops_validator_v5 (validator_version 5.0). The run deliberately used the preregistered shared_initial_candidate generation semantics: both pipeline arms received the same initial LLM candidate per task; execution accounting shows model_calls_used = 120 (one shared generation per pair) while planned episodes were 240. Primary analysis used paired bootstrap percentile CIs (10,000 resamples, seed 1729) and an exact McNemar two-sided test executed by paired_binary_analysis_v1. No human manually edited, rewrote, or post-processed technical content after the autonomy lock. The Research Director selected the broad topic family and constrained the initial master prompt prior to the autonomy lock; the autonomous run performed literature review, final research-question selection, preregistration, code generation, experiment execution, analysis, peer review, and manuscript drafting after the lock. Immutable reference to the initial master prompt: provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). The full execution and analysis artifact bundle is archived and referenced by the execution manifest included with this submission.

REFERENCES

- [1] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, David Walker, "Don't Mind the Gap", 2016, 10.1145/2934872.2934909
- [2] Radu Stoenescu, Matei Popovici, Lorina Negreanu, Costin Raiciu, "SymNet", 2016, 10.1145/2934872.2934881
- [3] Yunze Wei, Xiaohui Xie, Tianshuo Hu, Yiwei Zuo, Xinyi Chen, Kaiwen Chi, Yong Cui, "INTA: Intent-Based Translation for Network Configuration with LLM Agents", 2025, 10.1109/icnp65844.2025.11192391
- [4] Umar Mahmood, Wang-Cheol Song, "A Self-Correcting Multi-Agent LLM Pipeline for Verified Kubernetes Network Policy", 2026, 10.23919/ifipnetworking70592.2026.11578993
- [5] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318